DNSC 6330: Responsible Machine Learning*

Lecture 04: Robustness, Generalization, and Dataset Drift

Michael Akinwumi

Adjunct Professor of Responsible Machine Learning

Department of Decision Sciences

The George Washington University, School of Business

April 6 & 7, 2026

*This material is shared under a CC BY 4.0 license which allows for editing and redistribution, even for commercial purposes. However, any derivative work should attribute the author.

Outline (2.5 hours)

1. Recap of Lecture 03
2. Generalization: Theory and Failure Modes
3. 5 min Break
4. Distribution Shift: Taxonomy and Mechanics
5. 5 min Break
6. Stress Testing and Robustness Evaluation
7. 10 min Quiz
8. Applied Robustness Testing
9. Homework

Recap: From Bias to Robustness

Established in Lecture 03

- Disparate impact does not require intent.
- AIR, ME, SMD each capture a different dimension of disparity.
- The Impossibility Theorem is a *value choice*, not a bug.
- Intersectional testing surfaces harms invisible to aggregate metrics.
- Measurement without governance is just a report.

But ...

- What happens **over time**?
- What happens when the environment changes?
- What happens when models fail silently?

Recap: From Bias to Robustness

Core Question

Will your model still be responsible post-deployment? If yes, for how long?

Bridge to Today

1. Bias measurement reveals *current* disparity.
2. Robustness testing asks whether the model *remains reliable* as conditions change–a prerequisite for defensible deployment.

Recap: From Bias to Robustness

Learning Objectives

By the end of this lecture, you should be able to:

1. Distinguish robustness from generalization
2. Identify spurious correlations
3. Differentiate data drift from concept drift
4. Design stress tests for ML systems
5. Diagnose long-horizon failures

Generalization: The Problem

Generalization

- Performance on unseen but **similar** data
- Usually evaluated under the assumption that train and test distributions are close

Generalization: The Problem

Formal Setup

$\hat{h} : \mathcal{X} \to \mathcal{Y}$ trained on P_{train} where $P_{\text{train}} \subset \mathcal{X} \times \mathcal{Y}$, $\quad P_{\text{test}} \subset \mathcal{X} \times \mathcal{Y}$ *and* $\quad P_{\text{train}} \cap P_{\text{test}} = \varnothing$

Empirical risk:

$$R_{\text{emp}}(\hat{h}) = \mathbb{E}_{(x,y)\sim P_{\text{train}}}[\ell(\hat{h}(x), y)] \tag{2.1}$$

True risk:

$$R_{\text{true}}(\hat{h}) = \mathbb{E}_{(x,y)\sim P_{\text{test}}}[\ell(\hat{h}(x), y)] \tag{2.2}$$

Generalization: The Problem

Generalization gap

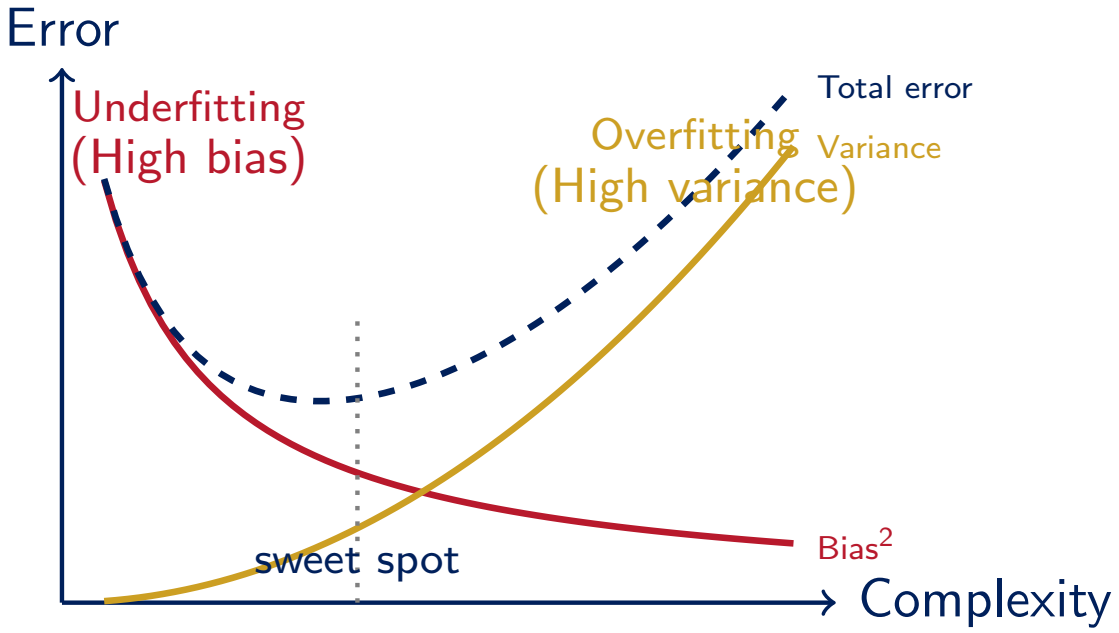
$$G_{\text{gap}}(\hat{h}) = R_{\text{true}}(\hat{h}) - R_{\text{emp}}(\hat{h}) \tag{2.3}$$

In the responsible ML setting, a large gap means the model's in-training performance *cannot be trusted in deployment*.

Why this matters operationally

- Credit-scoring models trained in a boom may fail in a downturn.
- Fraud detectors trained on 2019 patterns miss 2024 fraud tactics.
- Hiring models trained on past hires encode past biases *and* past economic conditions.

Generalization: Spurious Correlations



Governance implication

High-variance models that perform well in validation but fail in production create *audit-trail gaps*.

Bias–Variance Tradeoff

- **Underfitting (high bias):** Model too simple to capture real patterns.
- **Overfitting (high variance):** Model memorizes training quirks.
- Neither is safe for high-stakes deployment.

Generalization: Spurious Correlations

Spurious Correlations: When the Model Learns the Wrong Thing

A **spurious correlation** is a statistical association in the training data between a feature X_j and the label Y that *does not reflect a causal or stable relationship*, but rather a confound specific to the training sample or collection context.

Generalization: Spurious Correlations

Classic examples

- **Tank detection (apocryphal):** CNN learned *sunny vs. cloudy sky*, not tank vs. no tank. See "The Neural Net Tank Urban Legend" and Jeff Kaufman's "Detecting Tanks."
- **Medical imaging:** Chest X-ray model learned from scanner manufacturer, not disease signal (Zech et al., 2018).
- **NLP:** Sentiment model relied on "not" token position rather than semantic content.

Generalization: Spurious Correlations

Credit scoring analog

- Model trained 2015–2019 associates stable employment with good credit during economic expansion.
- COVID-19 shock breaks this: furloughed workers with excellent credit are mis-scored as high-risk.
- **The correlation was real in train; absent in deploy.**

Detection strategy

Feature-importance audits (SHAP) + domain-expert review flag features that should not be predictive.

Generalization: Shortcut Learning

Geirhos et al. (2020) — *Shortcut Learning in Deep NNs*

Models exploit "shortcuts" — features that correlate with labels in training but fail to reflect the intended task. Three levels of shortcut learning:

1. **Dataset shortcuts:** Label leaks through metadata, scanner ID, demographic proxy.
2. **Task shortcuts:** Easier proxy task substituted for harder true task.
3. **Deployment shortcuts:** Context cues present in training absent in target domain.

Generalization: Shortcut Learning

Why shortcuts are a governance problem

- Standard cross-validation *cannot detect* shortcuts if train and test share the same confound.
- Shortcuts can encode protected attributes (proxy discrimination from Lecture 03).
- A high-accuracy model may be making *correct predictions for entirely wrong reasons*.

Diagnostic

Subgroup accuracy on attribute-stratified slices + counterfactual perturbation testing.

Break

5 minutes

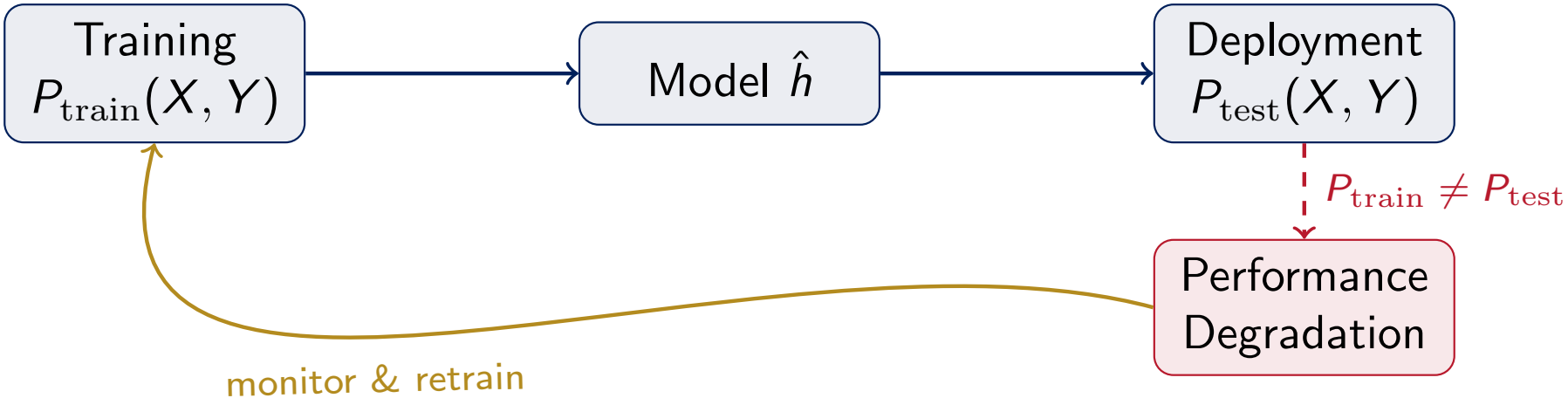
Back at [time + 5 min]

We will pick up with: Distribution Shift: Taxonomy and Mechanics

Distribution Shift: The Problem

Core idea

A model is trained on data from distribution $P_{\text{train}}(X, Y)$. At deployment time, new observations are drawn from $P_{\text{test}}(X, Y)$. **Distribution shift** occurs when $P_{\text{train}} \neq P_{\text{test}}$.



Distribution Shift: The Problem

Types of shift

- **Covariate shift:** $P(X)$ changes; $P(Y|X)$ stable.
- **Label shift:** $P(Y)$ changes; $P(X|Y)$ stable.
- **Concept drift:** $P(Y|X)$ changes — the relationship itself evolves.
- **Dataset shift:** $P(X, Y)$ changes.

Why the distinction matters

Different shift types require different remedies: importance weighting, threshold recalibration, or full retraining. Mis-diagnosing shift type leads to ineffective or harmful corrections.

Distribution Shift: Covariate Shift vs. Concept Drift

Shift Type	$P(X)$	$P(Y \mid X)$	$P(Y)$	**Typical Remedy**
Covariate shift	Changes	Stable	May change	Importance weighting
Label / prior shift	Stable	Stable	Changes	Threshold recalibration
Concept drift	Any	**Changes**	Changes	Retraining; online learning
Sample selection bias	Biased	Stable	Stable	Collect representative data
Internal covariate shift	Layer inputs	N/A	N/A	Batch normalization

Distribution Shift: Covariate Shift vs. Concept Drift

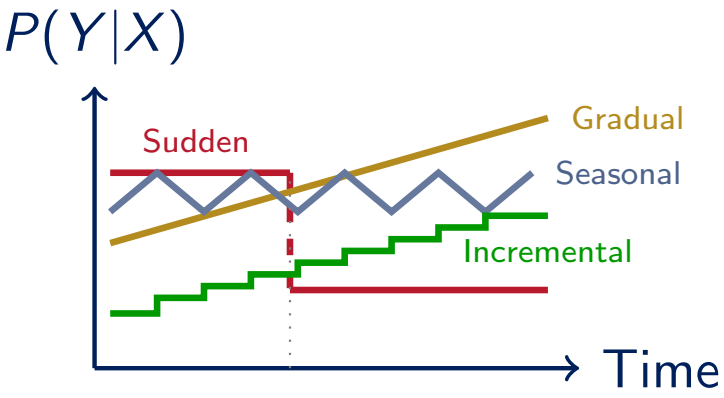
Credit scoring example

Covariate shift: applicant pool demographics shift after marketing campaign – same credit behavior, different population.
Concept drift: BNPL (Buy Now Pay Later) loans alter what prior payment history predicts about default.

Governance relevance

Concept drift can silently degrade *both* performance *and* fairness simultaneously – the model's learned relationship with a proxy may become more or less discriminatory as the world changes.

Distribution Shift: Concept Drift Patterns Over Time



Drift patterns

- **Sudden:** Regulatory change, pandemic shock, product policy change. Requires immediate retraining.
- **Gradual:** Slow demographic or behavioral change. Requires longitudinal monitoring.
- **Seasonal / recurring:** Cyclical patterns (*e.g.*, tax season, holiday spending). Requires temporal model segmentation.
- **Incremental:** Continuously evolving context (*e.g.*, evolving fraud tactics).

Distribution Shift: Statistical Tests for Input Distribution – P(X)

Population Stability Index (PSI)

$$\text{PSI} = \sum_{k=1}^{K} (p_k^{\text{tr}} - p_k^{\text{dep}}) \ln\left(\frac{p_k^{\text{tr}}}{p_k^{\text{dep}}}\right) \tag{4.1}$$

PSI < 0.10: stable; 0.10–0.25: monitor; > 0.25: retrain.

Kolmogorov–Smirnov test: Nonparametric two-sample test on score or feature distribution.

$$D = \sup_x |F_1(x) - F_2(x)| \tag{4.2}$$

Interpretation: (a) Larger $D \to$ greater distributional difference and (b)

Statistical test provides p-value for significance

Distribution Shift: Statistical Tests for Input Distribution – P(X)

Maximum Mean Discrepancy (MMD): Kernel-based two-sample test for high-dimensional feature space.

$$\begin{aligned}\mathrm{MMD}^2(X,Y) = \frac{1}{m(m-1)} \underbrace{\sum_i \sum_{j\neq i} k(x_i, x_j)}_{\text{A}} \\ - \frac{2}{m^2} \underbrace{\sum_i \sum_j k(x_i, y_j)}_{\text{B}} \\ + \frac{1}{m(m-1)} \underbrace{\sum_i \sum_{j\neq i} k(y_i, y_j)}_{\text{C}}\end{aligned} \tag{4.3}$$

Distribution Shift: Statistical Tests for Input Distribution – P(X)

Interpretation: One interpretation of the equation is that x_i's are the pre-deployment data points and y_i's are the post-deployment data points.

Output / performance tests – $P(Y|X)$

- **Rolling performance window:** Track AUC, F_1, calibration error over a sliding time window.
- **Page–Hinkley test:** Sequential change-point detection on cumulative prediction error. See *Francis, N., Ali, A. (2025).*
- **ADWIN (Adaptive Windowing):** Detects change in mean of a stream; commonly used in river/scikit-multiflow.
- **Chi-square test on score bins:** Compare score distribution across time buckets.

Distribution Shift: Statistical Tests for Input Distribution – P(X)

Practical note

Input tests are leading indicators (detectable *before* harm). Output tests are lagging indicators (detectable only after harm has occurred). A monitoring stack should include both.

Break

5 minutes

Back at [time + 5 min]

We will pick up with: Stress Testing and Robustness Evaluation

Robustness: The Problem

- Performance under **perturbation, stress, or shift**
- Asks whether the model remains reliable when conditions change

A model can generalize well and still fail catastrophically under shift.

Robustness

A model is **robust** if its performance degrades gracefully – not catastrophically – when inputs deviate from the training distribution, either through noise, shift, or adversarial perturbation.

Robustness: The Problem

Why Responsible ML Requires Robustness

A model is not responsible merely because it performs well on a held-out test set.

Responsible ML requires evidence that the system:

- remains reliable outside narrow lab conditions
- does not collapse under realistic perturbations
- does not silently degrade as data-generating conditions change

Operational implication: robustness is not an optional enhancement. It is part of defensible deployment.

Robustness: Stress Testing

Standard evaluation is insufficient

Holdout and cross-validation measure average performance on the *training distribution*. They do not measure:

- Behavior under worst-case subpopulations.
- Sensitivity to small, realistic perturbations.
- Performance under conditions not seen in training.
- Failure modes at deployment boundaries.

Robustness: Stress Testing

Stress testing in regulated industries

- **Banking:** Federal Reserve *Dodd-Frank Act Stress Test (DFAST)* / *Comprehensive Capital Analysis and Review (CCAR)* macroeconomic stress scenarios.
- **Insurance:** Catastrophe models tested against tail-risk scenarios.
- **Healthcare:** FDA guidance on AI/ML-based software device performance under distribution shift.
- **Hiring:** EEOC audit studies = stress testing ML claims in adversarial settings.

Robustness: Stress Testing

Slice-based evaluation (Slicing Criteria)

Partition the evaluation dataset into subsets (slices) defined by meaningful predicates over features:

$$\mathcal{S}_k = \{(x, y) \in \mathcal{D}_{\text{eval}} : \phi_k(x) = \top\}. \tag{6.1}$$

Compute performance metrics separately for each $\mathcal{S}_k$. Failure on a slice is a failure mode even if aggregate performance is strong. NOTE:

- $\phi_k(x)$: a predicate defining the slice
- Examples:
  1. income $<$ \$40K
  2. first-time homebuyer
  3. ZIP code in region X
  4. loan amount $>$ threshold

Robustness: Stress Testing

Slicing criteria $\phi_k(x)$ for credit scoring

- Loan vintage (year of origination).
- Loan type (mortgage, auto, personal).
- State / geography.
- Income quintile.
- First-time borrower vs. repeat borrower.
- Score bucket (*e.g.*, FICO deciles).
- Protected-attribute subgroups (connects to Lecture 03).

Robustness: Stress Testing

What to measure per slice

- AUC, Gini, KS statistic.
- Calibration (Brier score).
- FPR and FNR (bias connection).
- PSI relative to reference period.
- Lift curve deciles.

Red flag

A slice accounting for 5% of the population but 30% of the losses is a *governance priority*, not a statistical footnote.

Robustness: Sensitivity Analysis

Individual Conditional Expectation (ICE)

For each instance $x^{(i)}$, the ICE curve plots $\hat{f}(x_{-j}^{(i)}, x_j)$ as x_j varies – holding all other features constant.

ICE reveals **heterogeneous effects**: the model may respond very differently across subgroups.

Sensitivity index

Variance of predictions as a function of one feature:

$$V_j = \mathrm{Var}_{x_j}\left[\mathbb{E}_{x_{-j}}[\hat{f}(X)]\right] \tag{6.2}$$

High V_j with low domain justification $\Rightarrow$ spurious reliance.

Robustness: Sensitivity Analysis

Stress-scenario inputs

Define macroeconomic or operational scenarios and stress-test the model's output:

- Unemployment increases by 3 percentage points.
- Property values fall 20%.
- Interest rates rise 200 bps.
- New product feature (BNPL) enters as new variable.

Applicable directly to credit-scoring models in DFAST style validation.

Robustness: Sensitivity Analysis

Governance artifact

Sensitivity analysis outputs should be documented in the Model Risk Management (MRM) file with business justifications for features with high sensitivity scores.

Quiz Break

10 minutes

Quiz 04

Covers material from Lectures 01–04.

Live Coding: Robustness, Generalization, and Dataset Drift

Objective

Evaluate whether our COMPAS models remain reliable beyond the training setting.

- Source code: instructor GitHub link
- Note: this is based on the previous three classes where we looked at logistic regression and gradient-boosted models trained on COMPAS Broward County cohort data.

Setup

We use:

- Logistic Regression (interpretable baseline)
- Gradient-Boosted Tree (higher-capacity model)

Live Coding: Robustness, Generalization, and Dataset Drift

- Same features and test set from prior lecture

Part A: Distribution Drift

- Compare train vs. test distributions
- Metrics: PSI, KS test, MMD
- Question: *Has the data generating process changed?*

Part B: Generalization & Spurious Correlations

- Measure train vs. test performance gap (AUC, accuracy)
- Inspect feature reliance (permutation importance)
- Counterfactual swaps (race, gender) $\rightarrow$ prediction shifts
- Question: *Is the model learning stable signals or shortcuts?*

Live Coding: Robustness, Generalization, and Dataset Drift

Part C: Robustness & Stress Testing

- Slice-based evaluation (race, gender, age, charge type)
- Scenario stress: perturb `priors_count`
- Sensitivity analysis (ICE curves, variance of predictions)
- Question: *Does performance degrade gracefully under change?*

Key Takeaway

A model that performs well on a test set is **not** necessarily robust. Responsible deployment requires evidence of *stability under shift, stress, and subgroup variation*.

Homework Overview

Two assignments extend today's lecture from **measurement** to **audit-level reasoning**:

1. **Group theoretical homework:** build a sociotechnical audit framework for model reliability under drift, overfitting, spurious correlations, and robustness failure.
2. **Individual coding homework:** implement an end-to-end reliability audit in Python using the COMPAS pipeline from class.

Core principle

A model that performs well on average may still fail under shift, fail for subgroups, or fail for the wrong reasons.

Group Homework: Theoretical Audit, Due at 11:59pm ET on April 30, 2026

Title: *When Good Models Fail: A Sociotechnical Audit of Predictive Systems*

Objective:

- Develop a formal audit framework for predictive models under:
  - distribution drift
  - generalization failure
  - subgroup disparity
  - robustness stress

Your report must:

- distinguish covariate shift, label shift, and concept drift
- explain overfitting, shortcut learning, and spurious correlations
- connect metrics to governance decisions

Group Homework: Theoretical Audit, Due at 11:59pm ET on April 30, 2026

- explain why performance without monitoring is not sufficient

Group Homework: Required Components
Part A: Conceptual framework

- Define drift, generalization, spurious correlation, and robustness

Part B: Measurement → governance mapping

- Build a table linking:
  - risk type
  - metric
  - what it detects
  - failure mode
  - policy implication

Part C: Failure scenario analysis

Group Homework: Theoretical Audit, Due at 11:59pm ET on April 30, 2026

- Analyze a model with strong average performance but large subgroup error disparities

Part D: Less Discriminatory Alternative reasoning

- Propose two model or system changes and evaluate tradeoffs

Deliverable: 5–7 page report + one audit pipeline diagram

Individual Homework: Python Coding Audit, Due at 11:59pm ET on April 30, 2026

Title: *From Accuracy to Accountability: Stress Testing a Predictive Model*

Objective:

- Reproduce and extend the lecture pipeline in Python
- Use the COMPAS dataset and the models built in class
- Evaluate:
  - distribution drift
  - generalization
  - robustness
  - subgroup fairness

Submission:

- one clean, standalone Google Colab notebook

Individual Homework: Python Coding Audit, Due at 11:59pm ET on April 30, 2026

- runnable from top to bottom
- code + outputs + short markdown interpretations

Individual Homework: Coding Tasks

Part A: Distribution drift

- Compute PSI and KS for numeric features
- Compute MMD in encoded feature space
- Compare train vs. test score distributions

Part B: Generalization

- Compare train vs. test AUC, accuracy, and log loss
- Diagnose overfitting using performance gaps

Individual Homework: Python Coding Audit, Due at 11:59pm ET on April 30, 2026

Part C: Spurious-correlation probe

- Run counterfactual swaps on selected attributes
- Measure change in predicted probabilities

Part D: Robustness

- Stress test `priors_count`
- Produce ICE curves and sensitivity summaries

Part E: Slice-based evaluation

- Compare performance by race, gender, and age slices

Homework Evaluation Criteria

Group homework

- conceptual precision
- rigor of metric-to-governance mapping
- quality of sociotechnical reasoning
- clarity and defensibility of recommendations

Individual coding homework

- correctness of implementation
- code clarity and reproducibility
- interpretation quality
- evidence of audit-level reasoning

Homework Evaluation Criteria

What earns a strong grade

Do not just report metrics. Explain what they mean, what they miss, and what actions they justify.

Grading and Policy

- Grading
  - Weekly assignment: $\frac{2.5}{10}$
  - Public GitHub with working codebase: $\frac{2.5}{10}$
  - Weekly quiz: $\frac{2}{10}$
  - Capstone: $\frac{3}{10}$
- Coding/Scripting
  - Python
  - R
- Collaborative Infrastructure
  - GitHub

Grading and Policy

- Capstone project
  - 2024 Home Mortgage Disclosure Act (HMDA): Loan/Application Records (LAR) data
  - Public LAR data: dictionary
  - Goal: Develop a classification model that predicts `action_taken` and apply principles taught in class to the model.
  - Label creation: `action_taken` values 1 and 2 coded as 1 and 3 coded as 0 while records matching other values are filtered out.
  - Group (no more than 5 members and no less than 3 members)
  - Group members are randomly assigned by instructor using the standard *math* and *random* libraries in **Python**.
- Syllabus: link
- Officer hours: Funger Hall 415R, Fri, 4:00PM - 6:00PM
- Class resources:
  - Instructor's materials
  - Additional resources from Patrick Hall